

Université de Montréal

Département d'informatique et de recherche opérationnelle

Outil d'aide au calcul en génie structurel

Yamir Alejandro Poldo Silva

Superviseur : Pascal Archambault

Rapport remis dans le cadre du cours
Projet en informatique — *IFT3150-E25*

8 Août 2025

Table des matières

1 Introduction **3**
1.1 Contexte . . . 3
1.2 Problématique/Motivation . . . 4
1.3 Objectifs du projet . . . 4

2 Analyse des besoins et étude préliminaire **5**
2.1 Exigences fonctionnelles et non-fonctionnelles . . . 5
2.1.1 Exigences fonctionnelles . . . 5
2.1.2 Exigences non-fonctionnelles . . . 5
2.2 Méthodologie utilisée . . . 6
2.3 Étude de solutions similaires ou existantes . . . 6

3 Conception **7**
3.1 Architecture du système . . . 7
3.2 Modèle de données . . . 8
3.3 Choix technologiques . . . 9
3.4 Contraintes ou considérations spécifiques . . . 10
3.4.1 Exactitude des données et sécurité . . . 10

4 Implémentation **11**
4.1 Particularités vis-à-vis de la conception et autres contraintes . . . 11
4.2 Flux principaux . . . 12
4.3 Description détaillée des composants principaux . . . 13

5 Évaluation **17**
5.1 Tests unitaires et fonctionnels réalisés . . . 17
5.2 Évaluation de performance . . . 17
5.3 Évaluation d'utilisabilité . . . 18
5.4 Comparaison avec les objectifs initiaux . . . 19

6 Discussion critique **20**
6.1 Analyse critique de la solution développée . . . 20
6.2 Problèmes rencontrés et solutions mises en place . . . 20
6.3 Améliorations possibles . . . 20

7 Conclusion **22**
7.1 Bilan du projet . . . 22
7.2 Leçons tirées . . . 22
7.3 Pistes d'amélioration et développements futurs . . . 22

Remerciements **23**

Bibliographie **24**

A Annexes **25**
A.1 Extraits de code détaillés . . . 25
A.1.1 Fichier solver.py . . . 25
A.1.2 Fichier filtrage.py . . . 26
A.2 Captures d'écrans de l'application . . . 28
A.3 Jeux de tests, résultats d'évaluation complets . . . 31
A.3.1 Instructions d'exécution . . . 31
A.3.2 Test de performance . . . 31
A.3.3 Test unitaires sur solver (DSL) et filtrage . . . 32
A.3.4 `DSL de test` . . . 34
A.4 Documentation technique, schémas supplémentaires . . . 36
A.4.1 Documentation technique du DSL . . . 36
A.4.2 Lexique SST12 . . . 39
A.4.3 Licence SST12 (CISC) . . . 42

1. Introduction

1.1. Contexte

Dans le domaine du génie structurel, la modélisation et le calcul des contraintes mécaniques sur les éléments porteurs sont des étapes fondamentales de la conception pour garantir la sécurité et la fiabilité des structures. Ces calculs permettent d'identifier les efforts (traction, compression, flexion, torsion, cisaillement) que subissent les éléments tels que les poutres, colonnes ou d'autres éléments porteurs.

Pour résister efficacement à ces efforts, ces éléments sont souvent fabriqués sous forme de profilés, c'est-à-dire des pièces dont la section transversale est constante sur toute la longueur. Cette section peut prendre plusieurs formes (I, U, T, L, etc.) adaptés à des types spécifiques d'efforts. Les profilés sont conçus pour résister à divers efforts mécaniques tout en minimisant le poids et le coût.

Le choix du profilé dépend de plusieurs paramètres et contraintes :

— le type d'efforts mécaniques ;
— la distance entre les appuis (la portée) ;
— la répartition des charges et les déformations admissibles ;
— le mode de fabrication et assemblage ;
— le poids, l'encombrement et les contraintes environnementales ;
— les coûts des matériaux et de la main-d'œuvre.

La norme CSA S16 (9) régit le choix et l'utilisation des profilés en acier, en définissant les critères de sécurité, des méthodes de calcul et les valeurs limites avec marges de sécurité. Deux méthodes issues de cette norme doivent être modélisées et implémentées dans le système : Le **dimensionnement** d'une poutre et la **vérification** d'une poutre.

Dimensionnement (13) : Cette méthode consiste à déterminer, pour un élément donné, un profilé admissible répondant aux contraintes mécaniques selon les étapes suivantes :

1. Déterminer les charges applicables selon la position de l'élément.
2. Calculer les charges applicables.
3. Pondérer les charges selon le cas de chargement le plus critique.
4. Transférer les charges linéaire selon le cas de chargement applicable et calculer le moment maximal (Mmax) et et le cisaillement maximal (Vmax).
5. Dans l'équation deltaMax des cas de chargement, Isoler l'inertie (I) (pour les poutres en acier) [...] en le remplaçant par le critère de flèche des tableaux des flèches admissibles [...].
6. Identifier une poutre dans les tables ayant un moment résistant (Mr), un cisaillement résistante (Vr) et une inertie (I) supérieur aux valeurs trouvées.

Vérification(13) : Cette méthode consiste à s'assurer qu'un profilé donné est adéquat face aux contraintes identifiées.

1. Identifier les charges qui s'appliquent sur l'élément à vérifier.
2. Calculer ces charges et les transfert selon le type de charge qui s'applique sur l'élément ;

3. Pondérer les charges selon le cas de chargement le plus critique ;
4. Calculer Mmax,Vmax, deltaMax et les comparer avec les valeurs de l'élément à vérifier : Mr, Vr et la flèche admissible.

1.2. Problématique/Motivation

Les ingénieurs concepteurs doivent effectuer, pour chaque élément structurel, les calculs des contraintes mécaniques et sélectionner un profilé approprié à partir de bases de données de profilés standardisées. Ces tâches sont souvent redondantes, chronophages et sujettes à des erreurs.

Plusieurs logiciels commerciaux proposent des solutions de calculs. Cependant, ces logiciels sont difficiles à utiliser, gourmands en ressources informatiques et coûteux. En conséquence, plusieurs ingénieurs préfèrent utiliser des feuilles de calcul comme Excel et faire des recherches manuelles dans les bases de données de profilés. Évidemment, ce n'est pas optimal en temps ni efficacité.

Afin de rendre ce processus plus accessible et plus souple, l'approche suivante a été développée : permettre à l'utilisateur de décrire lui-même la logique de calcul, sans avoir à écrire de code source. Cette idée s'appuie sur la mise en place d'un langage spécialisé (DSL, ou *domain-specific language*)(3) simple, défini à l'aide de fichiers `YAML`, qui sont ensuite interprétés dynamiquement par le moteur de calcul.

Cela permet, par exemple, de modifier une méthode de vérification ou de dimensionnement directement dans un fichier de configuration, sans toucher au code Python. Ce mécanisme vise à donner plus de contrôle aux utilisateurs avancés qui ne sont pas nécessairement experts en programmation, mais souhaitent adapter les règles de calcul utilisées dans le système.

1.3. Objectifs du projet

Le projet vise la création d'une solution logicielle gratuite, open-source et facile d'utilisation, qui facilite le calcul des contraintes mécaniques et la sélection des profilés.
Les objectifs principaux sont :

— concevoir une application web accompagné d'une API(10), pouvant être installé et exécutée localement sur n'importe quel ordinateur ou serveur ;

— permettre à l'usager de saisir les paramètres clés tels que la distance entre les appuis, la distribution des charges, entres autres ;

— effectuer automatiquement les calculs des contraintes mécaniques pour différents types d'éléments (poutres, colonnes, etc.) selon les normes ;

— filtrer une sélection optimale de profilés respectant les contraintes calculées ;

— proposer une interface intuitive permettant une saisie simplifiée et un affichage clair des résultats : contraintes calculées et choix des profilés.

2. Analyse des besoins et étude préliminaire

2.1. Exigences fonctionnelles et non-fonctionnelles

2.1.1 Exigences fonctionnelles

Interface et gestion des projets : Le système doit offrir à l'utilisateur une interface complète et intuitive pour gérer ses projets de calcul. Après s'être authentifié, l'utilisateur doit pouvoir accéder à son espace personnel, créer un nouveau projet ou sélectionner un projet existant. L'interface doit guider l'utilisateur dans le choix d'une méthode de calcul, puis dans la saisie structurée des paramètres requis.

Affichage des résultats et sélection des profilés : Une fois les calculs exécutés, les résultats doivent être présentés de manière claire et exploitable. Le système doit proposer une liste de profilés filtrés selon les contraintes techniques calculées. L'utilisateur doit pouvoir ajuster les paramètres de filtrage (par exemple la sensibilité) et modifier les propriétés des matériaux afin d'adapter les résultats à ses besoins.

Historique et traçabilité : Le système doit conserver un historique détaillé de chaque projet, incluant les méthodes de calcul utilisées, les paramètres saisis, les contraintes calculées et les profilés sélectionnés. Toutes les actions importantes de l'utilisateur (calculs, sélections, exportations) doivent être enregistrées afin de permettre un suivi complet des décisions prises. Cet historique, consultable à tout moment, doit aussi pouvoir être exporté au format Excel (`.xlsx`) pour archivage ou documentation.

Validité des résultats : Le système doit garantir la conformité des résultats aux exigences de la norme CSA S16 (9). Les formules appliquées fournissent des résultats exacte, et le filtrage des profilés doit prendre en compte un seuil de tolérance sur le nombre de résultats affichés, afin d'assurer la qualité et la pertinence des recommandations.

2.1.2 Exigences non-fonctionnelles

Les exigences non-fonctionnelles du système concernent principalement la sécurité, la validité des résultats, fiabilité, utilisabilité, compatibilité, traçabilité, ainsi que la maintenabilité et l'extensibilité du code.

Sécurité : L'authentification des utilisateurs doit être sécurisée (mot de passe chiffré, token, https, ...), afin que les données utilisateurs soient protégées contre les accès non autorisés.

Fiabilité : Le système doit assurer l'intégrité et la persistance des données même en cas d'interruption inattendue. Les fichiers générés à l'exportation doivent être valides, complets et cohérents avec les données affichées à l'écran.

Utilisabilité : L'interface doit être intuitive, avec une navigation claire entre les étapes du projet (création, calcul, exportation, paramétrage, visualisation).

Compatibilité : L'application doit fonctionner de manière stable sur les navigateurs (Chrome, Firefox, Safari, Edge).

Maintenabilité et extensibilité : Le code doit être structuré de manière à faciliter l'ajout futur de fonctionnalités (ex. : nouvelles méthodes de calcul, formats d'export).

2.2. Méthodologie utilisée

Le développement du projet a suivi une approche itérative et incrémentale structurée en deux grandes phases complémentaires.

Dans un premier temps, une phase de prototypage frontend a été réalisée, en utilisant des données fictives. Cette étape avait pour objectif de concevoir et tester le flux d'utilisation de l'application sans un backend fonctionnel. Elle a permis d'établir une vision claire de la structure de navigation, des composants d'interface et surtout, traduire les méthodes normées en une séquence d'interactions logicielles cohérente. De plus, il était nécessaire de déterminer les responsabilités respectives de l'utilisateur et du logiciel.

Cette phase de prototypage a donc permis de valider le flux d'utilisation de chaque méthode, de traduire les étapes normées en séquences interactives, et ce concevoir une interface utilisateur structurée. Ce choix a permis de valider les besoins utilisateurs tôt dans le développement sans avoir un backend implémenté.

Une fois cette base fonctionnelle établie, le développement s'est poursuivi de manière modulaire et progressive. Chaque composant ou fonctionnalité a été implémenté d'abord côté backend, avec des tests unitaires ou ponctuels permettant de valider son bon fonctionnement isolé. Ensuite, la fonctionnalité a été intégrée au frontend, où des tests globaux ont été effectués pour valider le flux complet de l'application. Des ajustements ont parfois été nécessaires sur les composants précédents afin de s'assurer de la cohérence du système.

Cette méthode a permis de garantir une intégration continue, une validation incrémentale et une réduction des erreurs d'intégration, tout en favorisant une bonne visibilité sur l'avancement du projet.

Le développement a été réalisé individuellement, en utilisant des outils tels que GIT pour le contrôle de version, VSCode pour l'environnement de développement, des outils de tests comme Postman pour les requêtes API. L'organisation du travail a suivi une logique de gestion souple par tâches basée sur la progression fonctionnelle plutôt qu'un découpage rigide en sprints.

2.3. Étude de solutions similaires ou existantes

En complément de l'étude des normes, le développement s'est également appuyé sur des feuilles de calcul Excel existantes. Bien que leur flux d'utilisation n'est pas optimal, ces fichiers contiennent déjà plusieurs étapes de calcul partiellement automatisées, ce qui a permis de réutiliser certaines formules, de valider des résultats intermédiaires, et de guider l'implémentation dans le logiciel. De plus, en conservant une organisation proche de celles des feuilles de calcul, le système vise à ne pas trop dépayser les utilisateurs familiers de cet environnement, tout en leur offrant une interface plus intuitive et spécialisée.

3. Conception

3.1. Architecture du système

Présentation synthétique de l'implémentation

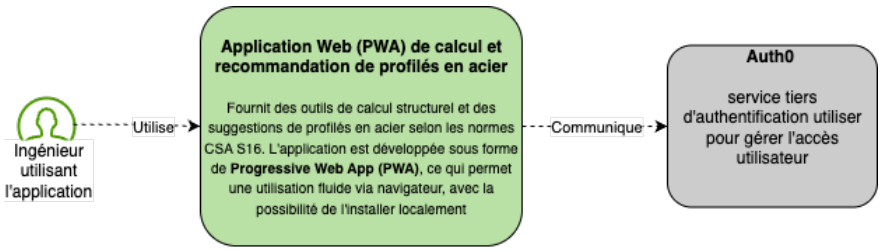


Figure 1 – Diagramme C1

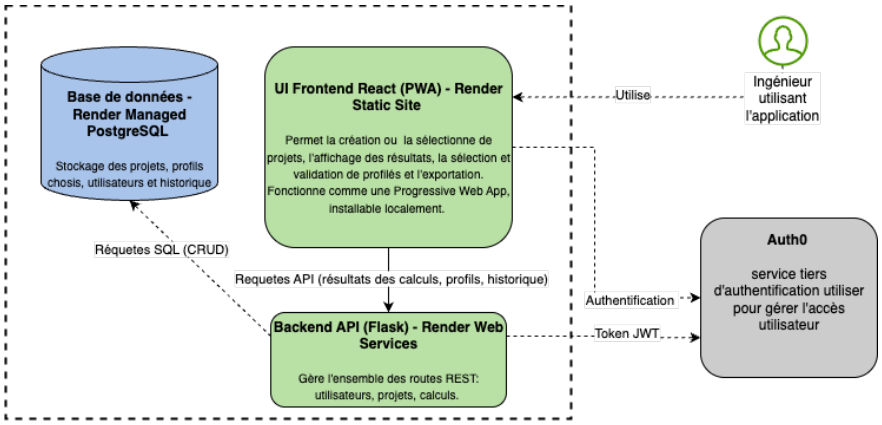


Figure 2 – Diagramme C2

Le système est principalement développé sous la forme d'une Progressive Web App (PWA) (1), accessible depuis un navigateur web avec la possibilité d'être installée localement par l'utilisateur. L'architecture est basée sur 2 composants logiciels principaux. Le frontend, développé avec React(11), correspond à l'interface utilisateur. Il gère la navigation, la saisie des paramètres de calcul, ainsi que la visualisation et l'export des résultats. Il fonctionne comme un Progressive Web App (PWA)(1). Le backend, conçu en Flask (Python)(4), expose des APIs REST(10) qui assurent les traitements des requêtes métiers : gestion des projets, calculs, enregistrements des résultats, etc.

Ces 2 composants communiquent avec des requêtes HTTPS(12) au format REST, avec des données sous forme de JSON.

La persistance des données (projets, profils sélectionnés, historiques, utilisateurs) est assurée par une base de données PostgreSQL hébergée sur Render, accessible depuis le backend.

L'authentification des utilisateurs est assurée par un service tiers Auth0(2). Ce dernier rend disponible un jeton JWT utilisé par le frontend pour sécuriser les requêtes envoyées au backend. Ce dernier sécurise les routes à l'aide d'un middleware qui valident la présence et l'intégrité du jeton.

3.2. Modèle de données

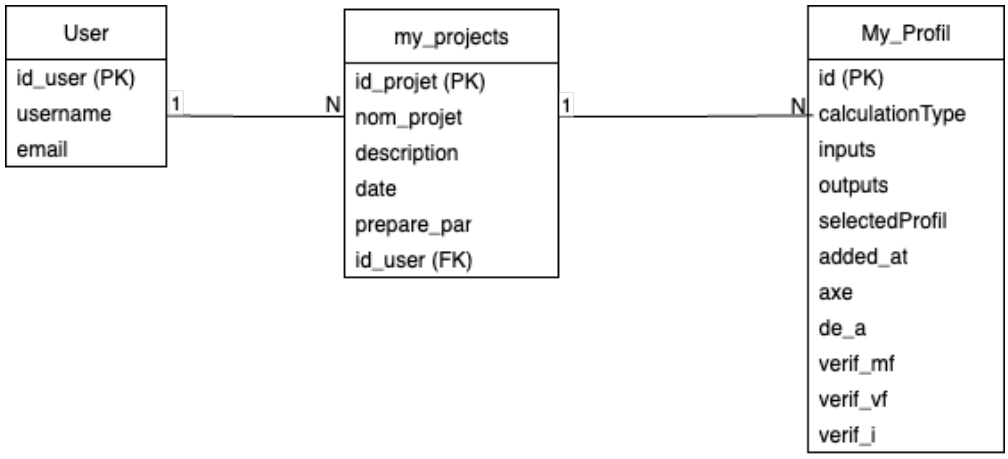


Figure 3 – Modèle entité-relation

La persistance des données est assurée par une base de données PostgreSQL. Les données structurés autour de 3 tables, résumés par le modèle relationnel ci-dessous :

user : contient les informations d'identification des utilisateurs.

— `id_user` (PK), `username`, `email`

my_projects : enregistre les projets associés à un utilisateur.

— `id_project` (PK), `nom_projet`, `description`, `date`, `prepare_par`, `id_user` (FK)

my_profils : contient les résultats des calculs associés à un projet.

— `id` (PK), `calculationType`, `inputs` (jsonb), `outputs` (jsonb), `selectedProfil` (jsonb), `added_at`, `axe`, `de_a`, `verif_mf`, `verif_vf`, `verif_i`, `id_project` (FK)

Les colonnes inputs, outputs et selectedProfil sont stockés au format jsonb. Cela permet de conserver des données semi-structurées compte tenu que les champs qui les composent sont variables d'une méthode à l'autre.

En complément de la base PostgreSQL, le système a accès à 3 fichiers contenant des données :

— Un fichier CSV contient l'ensemble des profilés d'acier standards avec leurs propriétés mécaniques. Il est utilisé lors du filtrage des profilés suite à un calcul.

— Un fichier YAML (config.yaml) contenant les paramètres globaux du système (valeurs par défaut, caractéristiques des matériaux, ect) modifiable depuis le frontend et utilisés lors des calculs et du filtrage.

— Un fichier YAML (templates.yaml) contenant les champs d'entrée et de sortie, ainsi que les formules de calcul. Le frontend l'utilise pour afficher dynamiques les champs d'entrée et sortie selon la méthode de calcul et le backend lors de la résolution des calculs (c'est une partie important du DSL de calcul).

3.3. Choix technologiques

Les choix technologiques ont évolué au fil du projet, suivant des ajustements à travers les différentes phases de prototypage et de validation. La version courante repose sur les technologies suivantes :

Composant	**Technologies**	**Justification**
Frontend	React, Vite, MUI, MUI X (DataGrid), XLSX	Architecture modulaire, rapidité de développement, composants UI modernes, export Excel côté client
Backend	Flask (Python), SymPy(6), psycopg	Simplicité de prototypage, calcul symbolique via DSL, communication directe avec PostgreSQL
Base de données	PostgreSQL (Render)	Support du type `jsonb`, maîtrise de l'outil, bonne intégration avec Render
Authentification	Auth0	Sécurité renforcée, évite la gestion manuelle des mots de passe, intégration simplifiée
Déploiement	Render	Déploiement automatisé via GitHub, gratuité en développement, simplicité d'utilisation
Distribution	Progressive Web App (PWA)	Installation locale possible, accessibilité multiplateforme, alternative légère à Electron

Table 1 – Résumé des choix technologiques et justifications

Du côté frontend, **React** offre une structure modulaire robuste, facilitant la maintenance et l'évolution de l'interface. Les bibliothèques **MUI** et **MUI X** simplifient grandement la conception d'interfaces réactives, et le composant `DataGrid` est idéal pour la visualisation des résultats. **XLSX** permet quant à elle une exportation directe côté client sans faire appel au backend.

Pour le backend, le choix de **Flask** repose sur sa légèreté et sa simplicité, bien adaptées à un projet individuel. **SymPy**, noyau du DSL, permet d'évaluer dynamiquement des équations, ce qui rend le moteur de calcul modulaire et extensible. L'approche retenue est plus souple que celle de bibliothèques FEM comme PyNite. **psycopg** est utilisé pour sa proximité avec SQL natif, ce qui évite la lourdeur d'un ORM comme **SQLalchemy**.

La base de données **PostgreSQL** a été préférée pour sa stabilité, la prise en charge native du type `jsonb`, et sa compatibilité avec Render. Des alternatives comme **SQLite** ou **MySQL** ont été écartées pour des raisons de limitations techniques.

L'authentification est assurée via **Auth0**, un service externe fiable qui simplifie la gestion des comptes et sécurise les échanges sans exposer directement les mots de passes.

Enfin, le déploiement via **Render**(5) permet une intégration continue facile avec GitHub, et offre une version gratuite répond bien aux besoins du développement. Le choix d'une **PWA** offre une expérience fluide, multiplateforme, sans les lourdeurs d'un framework comme Electron. L'application peut ainsi être utilisée localement tout en restant légère à maintenir.

3.4. Contraintes ou considérations spécifiques

3.4.1 Exactitude des données et sécurité

Bien que le système ait été testé et que les méthodes de calcul s'appuient sur des équations reconnues, **aucune garantie d'exactitude des résultats ne peut être fournie**. En effet, je ne suis pas un professionnel du génie structurel, et **les résultats produits ne doivent en aucun cas être utilisés sans validation par une personne qualifiée**.

Afin de minimiser le risque d'erreur lié à l'implantation directe des formules dans le code, le système repose sur un **langage spécifique au domaine (DSL)**, dont les règles sont définies dans le fichier `templates.yaml`. Cette approche permet à un professionnel de configurer ou d'adapter les règles de calcul (formules, paramètres, unités) sans modifier le code source, ce qui améliore la traçabilité et la sécurité.

Il est important de noter que les données dans le fichier `templates.yaml` sont seulement dans un but développement, implémentation et test du DSL. Elle doivent être dans tous les cas validés par un professionnel.

En résumé, **le système se veut un outil d'aide au calcul, et non un substitut aux vérifications réglementaires (9) ou à la responsabilité d'un ingénieur**. L'utilisateur final est responsable de la configuration des formules et de la validation des résultats dans leur contexte d'usage réel.

Licence et droit d'usage

Le système repose sur un fichier de profilés extrait du **SST12**(7), publié par le *Canadian Institute of Steel Construction (CISC)*(8). Ce fichier est soumis à une licence d'utilisation restrictive, interdisant toute diffusion, modification ou usage commercial sans autorisation préalable.

Conformément à cette licence, l'application développée :

— ne doit pas être distribuée publiquement avec le fichier SST12 ;

— est destinée uniquement à un usage personnel ou interne ;

— requiert l'obtention d'une licence commerciale valide pour toute redistribution.

Les termes complets de la licence sont disponible en annexe pour consultation.

4. Implémentation

Le système a été développé sous forme d'une **application web progressive (PWA)**, articulée autour d'un frontend en *React* et d'un backend en *Flask*, communiquant via une API **REST sécurisée** en `JSON`. Cette architecture légère et modulaire favorise un déploiement multiplateforme, une évolutivité rapide et une séparation claire des responsabilités.

Frontend : L'interface utilisateur repose sur des composants React modulaires, assurant la saisie des paramètres, l'affichage dynamique des résultats et l'interaction avec les profilés. Les bibliothèques `MUI` et `MUI X` offrent un rendu réactif et une intégration fluide de composants complexes tels que les tableaux de données.

Backend : Le backend expose les routes API via `app.py`, en s'appuyant sur plusieurs modules spécialisés. Le moteur de calcul symbolique `solver.py` interprète dynamiquement les équations définies dans un fichier YAML à l'aide de `SymPy`. Le module `filtrage.py` utilisent les contraintes mécaniques calculées pour sélectionner les profilés admissibles. Les échanges avec la base `PostgreSQL` sont assurés par `db.py`, tandis que l'authentification via `Auth0` est gérée dans `server.py`.

Persistance des données : La persistance s'appuie sur une base `PostgreSQL` pour les utilisateurs, projets et résultats, ainsi que sur trois fichiers externes : `templates.yaml` (formules de calcul), `config.yaml` (paramètres globaux), et `profils.csv` (caractéristiques des profilés standard).

4.1. Particularités vis-à-vis de la conception et autres contraintes

Changement d'approche de calcul : PyNite, initialement envisagée pour les calculs FEM, s'est révélée inadaptée au dimensionnement car elle nécessite des propriétés géométriques connues a priori. Le projet a donc adopté un DSL basé sur SymPy, plus modulaire et extensible.

Gestion des données : Les fichiers YAML (config.yaml, templates.yaml) stockés localement ont facilité le prototypage mais devront migrer vers PostgreSQL pour un déploiement multi-utilisateur.

Base SST12 : Plusieurs adaptations ont été nécessaires : calcul des colonnes manquantes (Mf, Vf, I) à partir des propriétés matériaux, et extraction par script des paramètres de tri (poids, hauteur) encodés dans les noms de lignes. Le calcul des colonnes manquantes a présenté son lot d'imprévu compte tenu la méthode de calcul varie selon la classe du profilés qui est identifié par un simple astérisque dans la base de données.

Authentification : Le système étant destiné à un nombre restreint d'utilisateurs, une solution d'authentification locale avait initialement été mise en place. Toutefois, à mesure que les besoins ont évolué, il est devenu nécessaire d'adopter une approche plus robuste afin de renforcer la sécurité des données utilisateurs et des routes de l'API. C'est dans ce contexte qu'un service tiers, **Auth0**, a été intégré pour prendre en charge l'authentification. Cette transition a impliqué plusieurs ajustements côté code, notamment l'ajout d'une route de synchronisation permettant d'aligner les utilisateurs connectés via Auth0 avec ceux stockés dans la base de données `PostgreSQL`.

Contrainte Render : La version gratuite arrête les services après 15 minutes d'inactivité (redémarrage de 5 minutes). Solution temporaire : endpoint /api/warmup interrogé par cron job toutes les 14 minutes.

4.2. Flux principaux

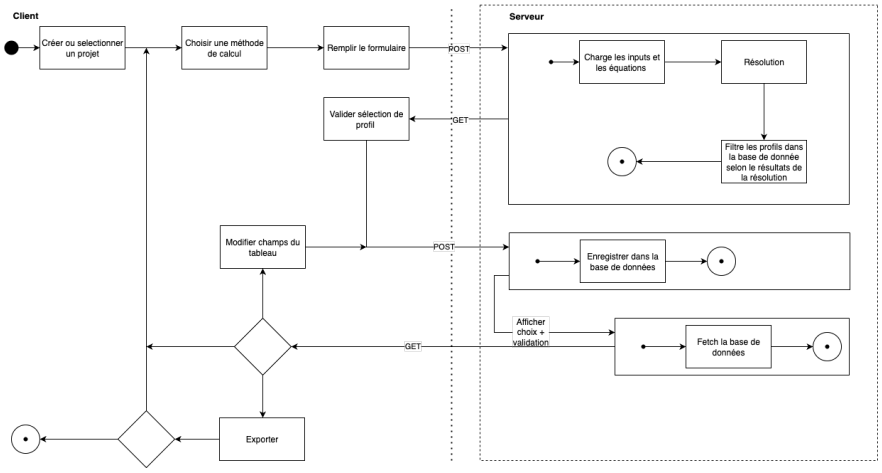


Figure 4 – Diagrammes d'activités pour les flux principaux

Le diagramme ci-dessus illustre les flux principaux d'activités du système, depuis la sélection d'un projet jusqu'à l'export des résultats. Il s'agit d'une vue générale de l'interaction entre l'utilisateur, le frontend et le backend. Certains éléments, tels que l'authentification avec Auth0, ont été volontairement omis afin de ne pas alourdir le diagramme.

L'interaction commence lorsque l'utilisateur se connecte à l'application. Il peut alors sélectionner un projet existant ou en créer un nouveau. Une fois un projet sélectionné, l'utilisateur choisit une méthode de calcul à partir d'un menu déroulant. Cette sélection déclenche l'affichage dynamique d'un formulaire, adapté à la méthode choisie.

Après avoir saisi les paramètres requis, l'utilisateur lance le calcul. Cela envoie une requête au backend contenant la méthode choisie ainsi que les paramètres. Le backend effectue alors les opérations suivantes :

— Il résout les équations de la méthode à l'aide d'un DSL (`solver.py`) ;

— Il filtre et trie les profilés standards selon les résultats obtenus et certaines contraintes mécaniques ou économiques (`filtrage.py`).

Le frontend reçoit ensuite :

— Les résultats du calcul, affichés à l'utilisateur pour analyse ;

— Une liste de profilés compatibles avec leurs propriétés.

L'utilisateur peut alors sélectionner un profilé et valider son choix. Ce choix est enregistré dans la base de données, avec l'ensemble des paramètres saisis, la méthode choisie et les résultats obtenus.

Finalement, le choix est affiché dans un tableau interactif. L'utilisateur peut alors :

— Reprendre le flux pour un autre calcul ;

— Modifier certaines valeurs du tableau ;

— Exporter les résultats sous forme de fichier `.xlsx`.

4.3. Description détaillée des composants principaux

Composants principaux du frontend

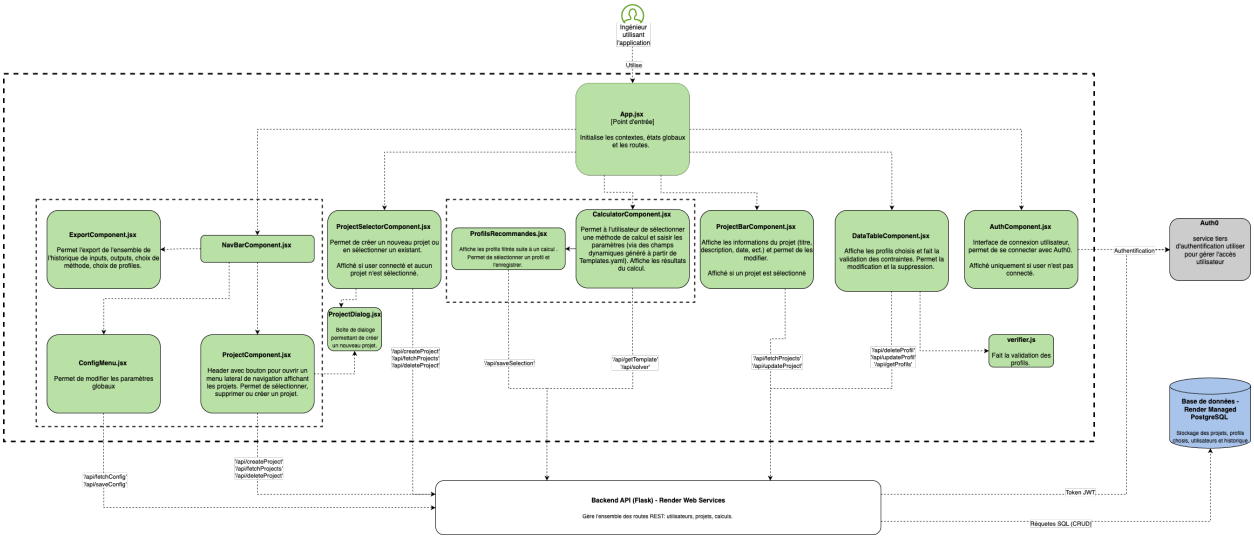


Figure 5 – C4 Model - Niveau 3 - Composant principaux du frontend

CalculationComponent.jsx Ce composant est responsable de la gestion et de l'affichage dynamique des formulaires de calcul. Lors de son initialisation, il interroge l'API via l'appel `GET /api/getTemplate` afin de récupérer la liste des méthodes de calcul disponibles. L'utilisateur peut ensuite sélectionner une méthode dans un menu déroulant, ce qui met à jour dynamiquement les champs d'entrée (`inputs`) et de sortie (`outputs`) à l'aide des états `setInputs` et `setOutputs`.

Une fois les paramètres saisis, le clic sur le bouton « Calculer » déclenche la fonction `handleClickCalculer`, qui envoie les données (`inputs` et `calculationType`) au backend via une requête `POST /api/solver`. Le backend retourne alors les résultats du calcul ainsi qu'une liste de profilés recommandés, stockés dans `outputs` et affichés à l'écran. Les données sont ensuite transmises au composant `ProfilsRecommandes` pour traitement et validation.

ProfilsRecommandes.jsx Ce composant affiche les profilés recommandés à l'aide du composant `DataGrid` de la bibliothèque MUI X. Il reçoit en paramètres les objets `inputs`, `outputs` et `calculationType`, et les affiche sous forme de tableau, incluant des colonnes telles que `Ds_m`, `Mr`, `Vr` ou encore `Ix`. L'utilisateur peut sélectionner un profilé et valider son choix, ce qui ouvre une boîte de dialogue dans laquelle il est invité à saisir le nom de l'axe (`axe`) et la portée (`de_a`).

Lors de cette validation, la fonction `handleValider` est appelée. Elle transmet toutes les données pertinentes au backend via la route `POST /api/save-selection`, notamment le profilé sélectionné, le type de calcul, les entrées et sorties, ainsi que les informations de projet et d'axe. L'appel déclenche également `triggerUpdateProfils`, un hook `useContext` qui met automatiquement à jour la vue principale contenant la liste des profilés validés.

dataTableComponent.jsx Ce composant affiche les profilés déjà validés pour un projet donné. Lors de son chargement, il interroge l'API via la route `GET /api/get_profils?id_project=X` pour récupérer

les données associés. Les résultats sont affichés sous forme de tableau, avec des colonnes telles que `Axe`, `De/A`, `Mf`, `Mr`, etc. L'utilisateur peut également choisir d'afficher ou masquer d'autres colonnes selon ses besoins. Chaque ligne peut être modifiée (ex. : nom de l'axe, appuis) ou supprimée, et les champs de vérification (`verif_mf`, `verif_vf`, `verif_i`) sont automatiquement calculés à l'aide du module `verifier.js`.

verifier.js Ce module JavaScript exporte une fonction chargée de vérifier localement si un profilé respecte les contraintes mécaniques. Il reçoit en paramètre les valeurs `mf`, `mr`, `vf`, `vr`, `i` et `ix`, et retourne trois booléens correspondant aux trois conditions : `verif_mf` (vrai si `mf / mr < 1`), `verif_vf` (vrai si `vf / vr < 1`) et `verif_i` (vrai si `ix / i < 1`). Ces vérifications permettent de s'assurer que le profilé satisfait aux contraintes de moment fléchissant, de cisaillement et d'inertie.

ExportComponent.jsx Ce composant permet d'exporter les résultats liés au projet courant. Lorsqu'un utilisateur clique sur le bouton « Exporter », une boîte de dialogue s'affiche, permettant de modifier certaines informations du projet (comme son nom ou sa description) et de sélectionner les données à inclure dans l'export. Les données sont ensuite formatées et exportées au format `.xlsx` à l'aide de la bibliothèque `xlsx`, sans traitement côté serveur.

Composants principaux du backend

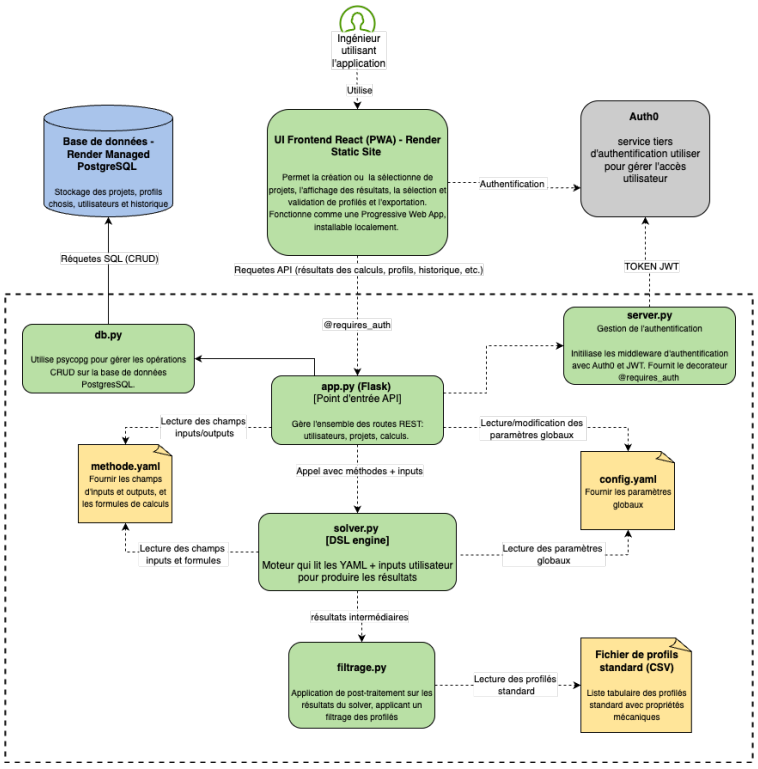


Figure 6 – C4 Model - niveau 2 - Backend

solver.py et DSL Ce module constitue le cœur du moteur de calcul symbolique. Il repose sur une approche orientée *DSL* (Domain-Specific Language), permettant de définir dynamiquement les méthodes de calcul à l'aide d'un fichier de configuration externe : `templates.yaml`. Ce dernier structure chaque méthode en trois parties : les paramètres d'entrée, les résultats attendus, et les équations de calcul.

Cette architecture dissocie complètement la logique de calcul du code Python. Il est ainsi possible d'ajouter ou de modifier une méthode en éditant simplement le fichier YAML, sans toucher au code source. Cela rend le système facilement extensible, même par des utilisateurs non programmeurs disposant d'une expertise dans le domaine du génie civil.

Le module `solver.py` charge dynamiquement les définitions fournies dans le DSL, construit un graphe de dépendances entre les formules, puis applique un tri topologique pour déterminer l'ordre d'évaluation. Les calculs sont ensuite effectués symboliquement à l'aide de `SymPy`.

L'algorithme `solve(methode, input_val)` suit les étapes suivantes :

1. **Chargement des données :** Les formules et variables de sortie correspondant à la méthode sélectionnée sont extraites de `templates.yaml`. Les paramètres globaux (comme les constantes de matériaux) sont chargés depuis `config.yaml`.
2. **Construction du graphe de dépendances :** Chaque équation est analysée avec `SymPy` pour identifier les variables nécessaires à son évaluation.
3. **Tri topologique :** Un tri topologique est appliqué afin de définir un ordre valide d'évaluation. À chaque itération, le système vérifie si les dépendances sont satisfaites ; sinon, une erreur est levée.
4. **Évaluation symbolique :** Les formules sont ensuite évaluées numériquement par substitution, produisant les résultats finaux de manière structurée.

voici un exemple synthétique du DSL utilisant la syntaxe YAML :

```
calculationOptions:
  Nom de la méthode :
    - name: x
      label: x (unité)

resultsTemplate:
  Nom de la méthode :
    - name: y
      label: y (unité)

formules:
  Nom de la méthode:
    formule_simple:  "x * x"
    formule_conditionnelle: Piecewise((expression1, condition1),
        (expression2, condition2), ..., (default_expr, True))
```

Grâce à cette architecture modulaire, l'ensemble des résultats produits par le moteur de calcul sont immédiatement exploitables par les modules suivants, notamment `filtrage.py`, chargé d'identifier les profilés qui satisfont aux contraintes calculées.

filtrage.py Le fichier `filtrage.py` contient la logique de sélection des profilés à partir des résultats du moteur de calcul.
L'algorithme `filter_sections(values)` fonctionne de la manière suivante :

1. **Chargement des données :** Le fichier CSV contenant les profilés est lu avec Pandas. Les champs numériques (Sx, D, T, W, Ix, Zx) sont normalisés et convertis.
2. Le moment résistant et l'effort tranchant nécessaire au filtrage doivent sont calculés selon les formules ci-dessous :
   — **Moment résistant :** `Mr = Phi × Zx × Fy / 1e6`
   — **Effort tranchant :** `Vr = Phi × 0.66 × Fy × D × W / 1000`
3. **Application des filtres :**

```
condition1 = df['Mr'] > Mf
condition2 = df['Vr'] > Vf
condition3 = df['Ix'] < In
```

4. **Extraction et tri :** Le champ `Ds_m` est parsé pour extraire la hauteur nominale et le poids linéique : `'W250x54' → hauteur: 250, poids: 54`. Ensuite, le tableau est trié selon ces critères pour suggérer les profilés les plus compacts.

La fonction retourne un tableau filtré et trié contenant uniquement les profilés qui satisfont aux contraintes.

5. Évaluation

5.1. Tests unitaires et fonctionnels réalisés

Les tests ont été mis en place afin de vérifier la robustesse du moteur de calcul symbolique (`solver.py`), la fiabilité du filtrage des profilés (`filtrage.py`), ainsi que la cohérence du DSL utilisé via `templates.yaml`. L'environnement de test repose sur `pytest` et couvre les composantes critiques du backend. De plus, des tests ponctuels ont été effectués avec **Postman** pour valider que les routes retournent des données cohérentes avec le backend.

Exemples de tests effectués :

- **Tests unitaires sur `solver.py`** :
  - Vérification des résultats (moment fléchissant, effort tranchant) avec une tolérance de 0.15 ;
  - Détection des erreurs en cas de méthode inconnue ;
  - Gestion d'expressions mal formées ou contenant des cycles de dépendance.
- **Tests fonctionnels sur `filtrage.py`** :
  - Vérification que les profilés retournés respectent les contraintes : $Mr > mf$, $Vr > vf$, $Ix < i$;
  - Cas limite : aucuns profilés retournés si les contraintes sont trop restrictives.
- **Tests d'intégration** :
  - Intégration entre `solve(...)` et `filter_sections(...)` pour valider le flux calcul $\rightarrow$ filtrage.
  - Test de formules conditionnelles utilisant `Piecewise` pour reproduire des cas de calcul complexes.

Bilan : Sur 15 tests automatisés, 15 ont été validés avec succès. Les tests effectués permettent une validation du système. Cela facilite l'ajout de nouvelles méthodes de calcul ou règle de filtrage.

5.2. Évaluation de performance

Les performances ont été mesurées sur un échantillon de tests unitaires et de tests de performance, exécutés sur 1000 itérations afin de simuler une utilisation intensive. Elle ont été effectuées sur un Macbook Pro, Apple Silicon M4, 16Go RAM.

Tests unitaires (temps moyens) : Les tests unitaires, qui valident des scénarios précis du moteur de calcul (`solver.py`) et du filtrage (`filtrage.py`), s'exécutent tous en moins de 0,05 s par itération. Par exemple :

- `test_solve_1` : 0,0405 s
- `test_solve_2` : 0,0226 s
- `test_filtrage_basique` : 0,0171 s
- `test_solver_filtrage` : 0,0302 s

Ces valeurs montrent que l'évaluation d'une méthode de calcul ou l'application d'un filtrage sur le CSV complet est réalisée dans un temps réaliste.

Tests de performance dédiés. Des scénarios intensifs ont été élaborés pour mesurer la rapidité du système sur un grand nombre d'itérations. Les résultats obtenus sont :

- `test_perf_solver` : 13,97 s pour 1000 itérations ($\approx$ 14 ms par appel)
- `test_perf_solver_piecewise` : 1,99 s pour 1000 itérations ($\approx$ 2 ms par appel)
- `test_perf_filtrage_csv_basique` : 2,52 s pour 1000 itérations ($\approx$ 2,5 ms par appel)
- `test_perf_integration_solve_plus_filtrage` : 17,15 s pour 1000 itérations ($\approx$ 17 ms par appel)

Analyse. Ces mesures indiquent que :

1. Le moteur de calcul (`solver.py`) est rapide, y compris avec des formules conditionnelles complexes (`Piecewise`).
2. Le filtrage sur un CSV complet reste performant.
3. L'exécution combinée `calcul + filtrage` reste inférieure à 20 ms par appel.

Ressources serveur et scalabilité. Le système est actuellement déployé sur la plateforme Render, avec la configuration suivante : **0,1 CPU** et **512 Mo RAM**. Malgré ces ressources limitées, le *backend* répond avec des temps de chargement très courts. Pour anticiper une demande croissante, Render propose des configurations allant jusqu'à **8 CPU** et **32 Go RAM** à un coût compétitif, ce qui permettrait de supporter un grand nombre d'utilisateurs simultanés sans dégradation des performances.

5.3. Évaluation d'utilisabilité

L'interface a été conçue pour qu'un utilisateur professionnel puisse la prendre en main rapidement. Le formulaire de saisie joue le rôle d'un fil conducteur naturel : lors de la première connexion, la seule action possible est la création d'un projet. Une fois cet élément créé, l'utilisateur accède à son espace personnel, où il est d'abord invité à choisir une méthode de calcul. Les autres fonctionnalités se révèlent progressivement au fur et à mesure des interactions, ce qui permet une découverte guidée et intuitive.

Lors des présentations auprès des utilisateurs cibles, ceux-ci ont généralement compris rapidement le fonctionnement du système et ont pu l'utiliser efficacement. Un point d'amélioration récurrent concerne la compréhension des noms de certains paramètres, variables et colonnes, qui peuvent varier en clarté selon les habitudes de travail de l'utilisateur. En effet, les abréviations utilisées sont arbitraires, tandis que l'utilisation de noms complets alourdirait visuellement l'interface. Pour résoudre ce dilemme, une documentation claire et centralisée des termes employés devra être proposée aux utilisateurs. Une partie de cette information est déjà disponible dans le lexique fourni par la base de données SST12 et les manuels de référence, mais il sera nécessaire de l'uniformiser et de la rendre facilement accessible depuis l'application. Une version préliminaire figure déjà dans la documentation technique.

Du point de vue des performances perçues, les modules du *backend* offrent des temps de réponse suffisamment rapides pour que la latence réseau et la communication HTTPS ne constituent pas un frein. Le seul moment où un léger temps de chargement est perceptible correspond au flux complet `calcul + filtrage`. Ce délai reste toutefois court et ne nuit pas à la fluidité de l'expérience utilisateur. Les autres interactions sont pratiquement instantanées, sans délai perceptible.

5.4. Comparaison avec les objectifs initiaux

L'application développée répond globalement aux objectifs définis en début de projet, tant sur le plan fonctionnel que non-fonctionnel. Le tableau 2 présente une synthèse des objectifs initiaux et leur atteinte.

Objectif	**État**	**Commentaire**
Concevoir une application web avec API installable et exécutable localement	Atteint	L'application fonctionne via un backend Flask et un frontend React, pouvant être exécutés localement ou déployés sur un serveur.
Permettre à l'usager de saisir les paramètres clés (portée, charges, etc.)	Atteint	Un formulaire interactif guide l'utilisateur dans la saisie des paramètres requis pour la méthode de calcul choisie. Les unités et labels sont précisés pour limiter les erreurs d'entrée.
Effectuer automatiquement les calculs selon les normes	Atteint	Le moteur `solver.py` exécute les formules stockées dans un DSL YAML, avec validation et gestion des dépendances. Les calculs respectent les critères de la norme CSA S16.
Filtrer une sélection optimale de profilés	Atteint	Le module `filtrage.py` applique des critères de résistance (Mr, Vr, Ix) et renvoie les profilés conformes. Les résultats sont triés et enrichis de dimensions nominales.
Proposer une interface intuitive pour la saisie et l'affichage	Atteint	Le flux d'utilisation est progressif : connexion → création de projet → choix de méthode → saisie → résultats. L'affichage est clair et adapté à un usage professionnel.
Historique, traçabilité et export des projets	Partiellement atteint	L'historique des projets est conservé et exportable en `.xlsx`. Les actions clés sont enregistrées, mais certaines données (timestamps détaillés, utilisateur à l'origine de chaque action) pourraient être enrichies dans une version future.
Sécurité et compatibilité	Atteint	Authentification gérée par Auth0 avec tokens sécurisés. Application testée sur Chrome, Firefox, Safari et Edge.
Maintenabilité et extensibilité	Atteint	Architecture modulaire (séparation backend et frontend, DSL YAML pour les méthodes). Facilité d'ajout de nouvelles méthodes de calcul ou formats d'export.

Table 2 – Comparaison entre les objectifs initiaux et les résultats obtenus

Dans l'ensemble, les fonctionnalités principales ont été réalisées conformément aux objectifs, avec une implémentation robuste et performante. Les tests unitaires et fonctionnels confirment la fiabilité du moteur de calcul et du filtrage. Les points d'améliorations concernent surtout des aspects d'enrichissement de la traçabilité, qui pourront être ajoutés lors de futures itérations.

6. Discussion critique

6.1. Analyse critique de la solution développée

Dans l'ensemble, la solution développée atteint les objectifs initiaux. L'implémentation d'un DSL constitue une approche originale tout en restant pragmatique. Elle présente l'avantage d'être modulaire et extensible, ce qui facilite l'évolution du système à long terme.

L'interface répond globalement aux besoins des utilisateurs et a été pensée pour évoluer en fonction de leurs attentes futures. Toutefois, plusieurs ajustements sont nécessaires du point de vue strictement UI et accessibilité. En l'état actuel, l'application n'est pleinement utilisable que sur des écrans d'au moins 10 pouces, et pour un confort optimal, un écran de 27 pouces est recommandé. L'utilisation sur un smartphone est impossible dans la version actuelle.

6.2. Problèmes rencontrés et solutions mises en place

La première implémentation du DSL était minimaliste : elle se résumait à 4 lignes de code évaluant directement un fichier YAML :

```
for methode, formula in formulas.items():
    context[methode] = sympify(
        formula,locals={"Max": Max, "Min": Min, "Piecewise": Piecewise})
            .evalf(subs=context)
```

En surface, cette méthode semblait efficace, mais elle a rapidement montré ses limites :

— Elle ne gérait pas les dépendances entre opérations : si l'opération 1 dépendait de l'opération 2, mais que celle-ci n'était pas encore calculée, l'opération 1 restait indéfinie.

— En cas de cycle de dépendances (ex. : opération 1 dépend de 2, et inversement), le système ne le détectait pas et échouait à évaluer les formules sans faire remonter l'erreur.

La version actuelle de `solver.py` corrige ces problèmes en construisant un graphe de dépendances entre les variables, effectuant un tri topologique pour déterminer l'ordre optimal de calcul et ajoutant des clauses `try/catch` pour détecter les formules invalides ou les cycles de dépendances.

Si cette version est plus robuste et maintenable, elle pose néanmoins une question d'usage : l'idée initiale était de permettre à l'utilisateur de définir ses propres méthodes de calcul. Cela impliquait qu'un professionnel qualifié en valide la justesse, lui transférant ainsi la responsabilité en matière de sécurité. Or, si l'utilisateur fournit une méthode mal formatée ou erronée, à qui incombe la responsabilité ? Ce point nécessite un cadre d'utilisation clair.

6.3. Améliorations possibles

Face aux limites actuelles du DSL, plusieurs pistes sont envisageables :

Utilisation d'un LLM (Large Language Model). L'idée d'utiliser un modèle comme Mistral ou LLaMA a été envisagée, mais rapidement écartée, car ces systèmes sont non déterministes : ils ne garantissent ni la reproductibilité, ni la conformité aux normes.

Formulaire guidé. Une alternative plus pragmatique serait d'intégrer un formulaire interactif guidant l'utilisateur lors de l'implémentation de nouvelles méthodes. Cela permettrait de garantir un formatage correct et de détecter les erreurs avant validation, mais au prix d'une flexibilité réduite par rapport à la saisie libre en YAML.

Dans l'état actuel, l'implémentation de nouvelles méthodes se fait conjointement entre un utilisateur connaissant le système et un ingénieur expert du domaine.

Autres pistes d'amélioration. Pour de futures versions, plusieurs ajouts seraient intéressants :

— intégration d'outils graphiques permettant de visualiser les contraintes calculées,

— prise en charge de formats d'export supplémentaires : PDF, CSV, ou formats compatibles CAD,

— adaptation complète de l'interface aux différents formats d'écrans et aux interactions tactiles.

7. Conclusion

7.1. Bilan du projet

Dans l'ensemble, le projet peut être considéré comme un succès. Il a non seulement atteint les objectifs fixés, mais m'a également permis d'approfondir mes connaissances et de mettre à profit celles déjà acquises. Ce projet a représenté bien plus qu'un simple exercice de développement : plutôt que de privilégier uniquement des solutions simples et pragmatiques, j'ai choisi d'explorer de nouvelles méthodes, thématiques et technologies, enrichissant ainsi ma compréhension technique et conceptuelle.

7.2. Leçons tirées

J'ai appris que le rôle d'un informaticien dépasse largement les seules technologies utilisées. Il implique souvent un aspect pluridisciplinaire qui nous pousse à sortir de notre zone de confort et à nous intéresser à des domaines parfois éloignés de notre expertise initiale. Dans le cadre de ce projet, j'ai notamment dû me familiariser avec certains principes du génie structurel. Bien que mes connaissances dans ce domaine restent limitées, elles ont été indispensables à la réalisation de l'application.

7.3. Pistes d'amélioration et développements futurs

La suite naturelle du projet serait de développer une version native pour ordinateur, tout en perfectionnant la version *web app*. L'objectif serait de recourir à un framework ne nécessitant pas de moteur web, afin d'optimiser les performances et la portabilité.

Il sera également important d'améliorer le système d'historique afin de conserver un journal complet des actions effectuées par les utilisateurs, rendant ainsi l'application plus adaptée aux audits.

De plus, la méthode de stockage des fichiers YAML devra être repensée afin de permettre un usage multi-utilisateurs et de faciliter la modification des méthodes de calcul par les utilisateurs de manière autonome.

Enfin, d'autres fonctionnalités pourront être ajoutées progressivement, en fonction de l'évolution des besoins. L'application a été pensée dès le départ pour être évolutive, grâce à une architecture modulaire et à une séparation claire des responsabilités.

Remerciements

Je tiens à exprimer ma gratitude à Pascal Archambault, mon superviseur, pour ses conseils et son accompagnement tout au long de la réalisation de ce projet. Je remercie également mon père et mon frère, tous deux ingénieurs civils, pour m'avoir aidé à acquérir des connaissances en génie civil et pour avoir testé l'application.

Bibliographie

[1] Applications web progressives (pwa). https://developer.mozilla.org/fr/docs/Web/Progressive_web_apps. Consulté en juillet 2025.

[2] Auth0 documentation. https://auth0.com/docs. Consulté en août 2025.

[3] Domain-specific language. https://en.wikipedia.org/wiki/Domain-specific_language. Consulté en juin 2025.

[4] Flask documentation. https://flask.palletsprojects.com/en/stable/. Consulté en juin 2025.

[5] Render documentation. https://render.com/docs. Consulté en août 2025.

[6] Sympy documentation. https://www.sympy.org/en/index.html. Consulté en juin 2025.

[7] Canadian Institute of Steel Construction. Structural section tables, version 12.1 - metric properties, April 2022. Copyright © 2022, https://www.cisc-icca.ca.

[8] Canadian Institute of Steel Construction. *Handbook of Steel Construction*. Canadian Institute of Steel Construction, Markham, Ontario, Canada, 12 edition, 2023.

[9] Canadian Standards Association. *CSA S16-19 : Design of Steel Structures*. CSA Group, Mississauga, ON, 2019.

[10] Lokesh Gupta. What is rest ? — rest api tutorial. https://restfulapi.net/, 2025. Consulté en juin 2025.

[11] Meta Platforms, Inc. React documentation. https://react.dev/reference/react, 2025. Consulté en mai 2025.

[12] Mozilla Developer Network. Méthodes http. https://developer.mozilla.org/fr/docs/Web/HTTP/Reference/Methods, 2025. Consulté en juillet 2025.

[13] Sébastien Thivierge. Recueil de notes - structure : Hiver 2023, technologie du génie civil, 2023. Préparé par Sébastien Thivierge, ing., M.ing., enseignant, Cégep de Saint-Hyacinthe.

A. Annexes

A.1. Extraits de code détaillés

A.1.1 Fichier solver.py

```python
import yaml
from sympy import Max, Min, Piecewise
from sympy.parsing.sympy_parser import parse_expr

def solve(methode, input_val, config_path='ressource/config.yaml', template_path='ressource/Templates.yaml'):
    # Chargement des configurations globales
    with open(config_path, "r") as file:
        config = yaml.safe_load(file)
    # Chargement du template de formules et r sultats
    with open(template_path, "r") as file:
        template = yaml.safe_load(file)

    try:
        # R cup re les outputs et formules pour la m thode demand e
        outputs = template["resultsTemplate"][methode]
        formulas = template["formules"][methode]
    except KeyError:
        # Si la m thode n'existe pas dans le template, on l ve une exception
        raise Exception(f"Template non trouv e pour la m thode: {methode}")

    # Pr pare le contexte de calcul (variables connues)
    context = config.copy()
    context.update(input_val)

    # Cr ation de la liste des d pendances pour chaque formule
    dependencies = {}
    for var, formula in formulas.items():
        try:
            # Analyse la formule pour extraire ses variables d pendantes
            expr = parse_expr(formula, local_dict={"Max": Max, "Min": Min, "Piecewise": Piecewise}, evaluate=False)
            dependencies[var] = {str(s) for s in expr.free_symbols
                                 if str(s) in formulas or str(s) in context}
        except Exception as e:
            # Erreur de parsing de la formule
            raise Exception(f"Erreur lors de l'analyse de la formule '{var}': {e}")

    # Initialisation des ensembles pour le tri topologique
    toSetFormulas = set(formulas)
    toSetContext = set(context)
    order_stack = []

    # Tri des formules selon l'ordre de r solution des d pendances
    while toSetFormulas:
        progress = False
        for var in list(toSetFormulas):
            # Si toutes les d pendances de la formule sont connues, on peut la r soudre
            if dependencies[var].issubset(toSetContext):
                order_stack.append(var)
```

```
                toSetContext.add(var)
                toSetFormulas.remove(var)
                progress = True
        if not progress:
            # Si aucune formule n'a pu  tre  r solue , il y a un cycle dans les
    d pendances
            raise Exception(f"Cycle  d  t e c t  dans les d pendances des formules:
    {toSetFormulas}")

    # R solution des formules dans l'ordre  d  t e r m i n
    for var in order_stack:
        try:
            try:
                # Parse la formule  nouveau pour l' valuer
                expr = parse_expr(formulas[var], local_dict={"Max": Max, "Min":
    Min, "Piecewise": Piecewise})
            except Exception as e:
                # Erreur de parsing
                raise Exception(f"Erreur lors de l'analyse de la formule '{var}':
    {e}")
            # Substitue les valeurs connues et  value  la formule
            value = expr.subs(context).evalf()
            context[var] = float(value)
        except Exception as e:
            # Erreur lors de l'  valuation  num rique
            raise Exception(f"Erreur lors de l'  valuation  de '{var}': {e}")

    # Retourne le contexte enrichi avec les r sultats  calcul s
```

Listing 1 – solver.py – Moteur de calcul DSL

A.1.2 Fichier filtrage.py

```
import pandas as pd
import yaml
import re

def filter_sections(values, dataframe_path = 'ressource/
    CISC_StructuralSectionTables_SST12.1.csv', config_path = 'ressource/config.yaml
    '):
    # Chargement de la configuration globale (phi, fy, etc.)
    with open(config_path, 'r') as file:
        config = yaml.safe_load(file)

    phi = float(config.get('phi', 0.9))
    fy = float(config.get('fy', 350))

    # Chargement du tableau des profil s
    df = pd.read_csv(dataframe_path, sep=';')
    df = df.replace({float('nan'): None})

    # Nettoyage et conversion des colonnes num riques
    for col in ['Sx', 'D', 'T', 'W', 'Ix', 'Zx']:
        df[col] = (
            df[col]
            .astype(str)
            .str.replace(' ', '', regex=False)
            .str.replace(',', '.', regex=False)
            .replace(['', '*'], 'None')
```

```
            .astype(float)
        )

    # Calcul du moment r sistant Mr pour chaque profil
    Mr_list = []
    for i, row in df.iterrows():
        if row['Prp'] == '*':  # Classe 3
            if pd.notnull(row['Sx']):
                Mr = phi * row['Sx'] * 1e3 * fy / 1e6
            else:
                Mr = float('None')
        else:  # Classe 1 et 2
            if pd.notnull(row['Zx']):
                Mr = phi * row['Zx'] * 1e3 * fy / 1e6
            else:
                Mr = float('None')
        Mr_list.append(Mr)
    df['Mr'] = pd.Series(Mr_list, dtype='float64')

    # Calcul de l'effort tranchant r sistant Vr
    df['Vr'] = phi * 0.66 * fy * df['D'] * df['W'] / 1000

    # Fonction pour extraire la hauteur nominale et le poids lin ique    partir
    de Ds_m
    def parse_ds_m(ds_m):
        expr = re.match(r'[A-Za-z]+(\d+)[xX](\d+)',str(ds_m))
        if expr:
            hauteur_nominale = int(expr.group(1))
            poids_lineique = int(expr.group(2))
            return hauteur_nominale, poids_lineique
        return None, None

    # Affichage de debug (peut  tre  retir  en production)
    print(df['Mr'].head(10))
    print(df['Vr'].head(10))
    print(df['Ix'].head(10))

    # R cup ration des contraintes    respecter
    Mf = float(values.get('mf', 0))
    Vf = float(values.get('vf', 0))
    In = float(values.get('i', 0))

    # Filtrage selon les contraintes de moment, effort tranchant et inertie
    condition1 = df['Mr'] > Mf      # Mr doit  tre  sup rieur au moment
    fl chissant requis
    condition2 = df['Vr'] > Vf      # Vr doit  tre  sup rieur    l'effort
    tranchant requis
    condition3 = df['Ix'] < In      # Ix doit  tre  inf rieur    l'inertie
    maximale autoris e

    result = df[condition1 & condition2 & condition3]

    # Extraction des dimensions pour chaque profil  filtr
    dimension_liste = result['Ds_m'].apply(parse_ds_m)
    result['Hauteur_nominale'] = [t[0] for t in dimension_liste]
    result['Poid_lineique'] = [t[1] for t in dimension_liste]

    # Tri des r sultats par hauteur nominale puis poids lin ique
    result = result.sort_values(by=['Hauteur_nominale', 'Poid_lineique'])
```

```
80      return result
```

Listing 2 – `filtrage.py` – Filtrage des profilés

Fichier verifier.js

```
1  // Fonction de v rification des contraintes m caniques
2  export function verifier({ mf, mr, ix, i, vf, vr }) {
3
4    // Initialisation des r sultats de v rification
5    let verif_mf = null;
6    let verif_vf = null;
7    let verif_i = null;
8
9    // V rifie la contrainte de moment fl chissant (mf/mr < 1)
10   if (mf !== undefined && mr !== undefined && mr !== 0) verif_mf = mf / mr < 1;
11   // V rifie la contrainte d'inertie (ix/i < 1)
12   if (i !== undefined && ix !== undefined && ix !== 0) verif_i = ix / i < 1;
13   // V rifie la contrainte d'effort tranchant (vf/vr < 1)
14   if (vf !== undefined && vr !== undefined && vr !== 0) verif_vf = vf / vr < 1;
15
16   // Retourne les r sultats de v rification
17   return { verif_mf, verif_vf, verif_i };
18
19 }
```

Listing 3 – `verifier.js` – Validation

A.2. Captures d'écrans de l'application

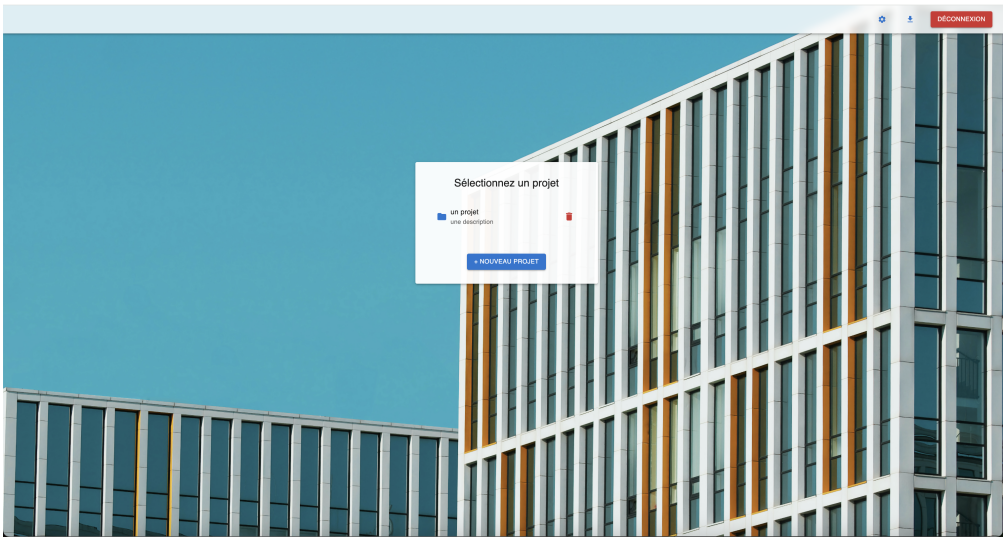


Figure 7 – Écran d'accueil

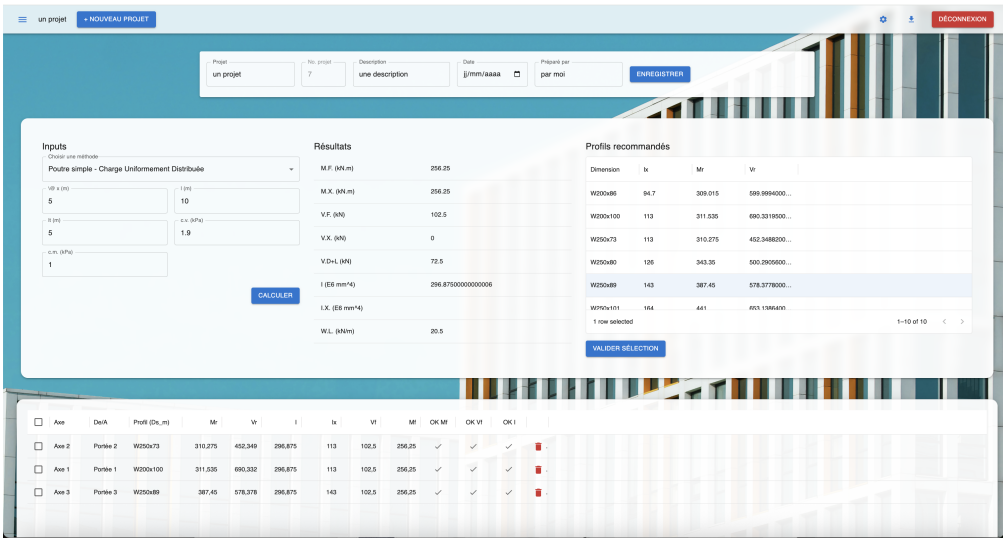


Figure 8 – Page principale

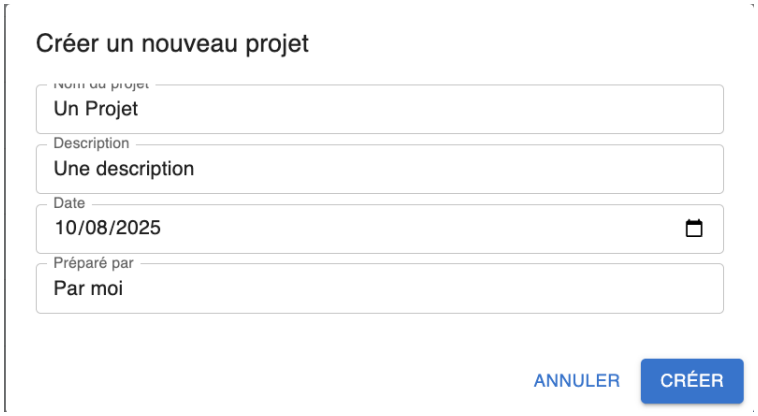


Figure 9 – Menu de création de projet

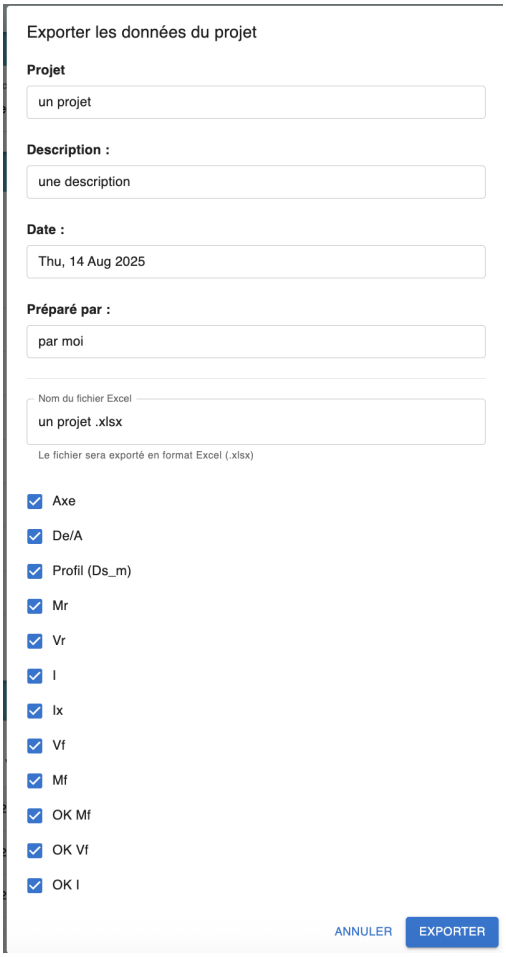


Figure 10 – Page d'export

A.3. Jeux de tests, résultats d'évaluation complets

Cette annexe regroupe les fichiers de tests utilisés pour valider le moteur de calcul (`solver.py`), le filtrage des profilés (`filtrage.py`), ainsi que des tests de performance :

- `tests/test_performance.py`
- `tests/test_dsl_et_filtrage.py`
- `ressource/test_dsl.yaml`

A.3.1 Instructions d'exécution

Lancer tous les tests : `pytest`

A.3.2 Test de performance

```python
import time
import pytest
from solver import solve
from filtrage import filter_sections

N = 1000

def _stats(durations):
    durations = sorted(durations)
    n = len(durations)
    mean = sum(durations) / n
    return mean

def _measure(func, *args, runs=N, **kwargs):
    durs = []
    for _ in range(runs):
        t0 = time.perf_counter()
        func(*args, **kwargs)
        durs.append(time.perf_counter() - t0)
    return _stats(durs)

@pytest.mark.performance
def test_perf_solver():
    inputs = {"l": 4.2, "lt": 1.0, "cv": 15.3, "cm": 9.0, "x": 1.0}
    mean= _measure(
        solve,
        "Poutre simple - Charge Uniformement Distribuee",
        inputs
    )
    print(f" solver     mean: {mean:.6f}s")
    assert mean < 0.050, f"solve trop lent (mean={mean:.6f}s)"

@pytest.mark.performance
def test_perf_solver_piecewise():
    inputs = {"x": 5}
    mean = _measure(
        solve,
        "Methode conditionnelle",
        inputs,
        template_path="ressource/test_dsl.yaml"
    )
    print(f" solver (piecewise)     mean: {mean:.6f}s")
```

```
    assert mean < 0.010

@pytest.mark.performance
def test_perf_filtrage_csv_basique():
    values = {"mf": 1000, "vf": 500, "i": 100000}
    mean = _measure(
        filter_sections,
        values,
        dataframe_path="ressource/test_sst12.csv"
    )
    print(f"filtrage(csv)     mean: {mean:.6f}s")
    assert mean < 0.030

@pytest.mark.performance
def test_perf_integration_solve_plus_filtrage():
    inputs = {"l": 10.0, "lt": 5.0, "cv": 1.9, "cm": 1.0, "x": 5.0}
    def end_to_end():
        res = solve("Poutre simple - Charge Uniformement Distribuee", inputs)
        filter_sections(res, dataframe_path="ressource/test_sst12.csv")

    mean = _measure(end_to_end)
    print(f" end-to-end     mean: {mean:.6f}s")
    assert mean < 0.050
```

Listing 4 – Test de performance

A.3.3 Test unitaires sur solver (DSL) et filtrage

```
import pytest
import os
from solver import solve
from filtrage import filter_sections

# test sur le solve
def test_solve_1():
    inputs = {"l": 4.2, "lt":1.0, "cv": 15.3, "cm": 9.0, "x": 1.0}
    results = solve("Poutre simple - Charge Uniformement Distribu e", inputs)
    ## On travaille une tolerance de 0.15
    assert abs(results["mf"] - 75.41) < 0.15
    assert abs(results["mx"] - 54.72) < 0.15
    assert abs(results["vf"] - 71.82) < 0.15
    assert abs(results["vd"] - 51.03) < 0.15
    assert abs(results["vx"] - 37.62) < 0.15
    assert abs(results["i"] - 35.42) < 0.15

def test_solve_formule_inconnue():
    inputs = {"l": 1, "lt": 1, "cv": 1, "cm": 1, "x": 1}
    with pytest.raises(Exception) as excinfo:
        solve("Methode inconnue", inputs)
    assert "Template non trouv e pour la m thode" in str(excinfo.value)

def test_solve_2():
    inputs = {"l": 10.00, "lt":5.0, "cv": 1.9, "cm": 1.0, "x": 5.0}
    results = solve("Poutre simple - Charge Uniformement Distribu e", inputs)
    ## On travaille une tolerance de 0.15
    assert abs(results["mf"] - 256.25) < 0.15
    assert abs(results["mx"] - 256.25) < 0.15
    assert abs(results["vf"] - 102.50) < 0.15
```

```
31      assert abs(results["vd"] - 72.5) < 0.15
32      assert abs(results["vx"] - 0.00) < 0.15
33      assert abs(results["i"] - 296.88) < 0.15
34
35  # Test sur le DSL
36  def test_dsl_cycle():
37      with pytest.raises(Exception) as excinfo:
38          solve("Methode avec cycle", {"x": 1}, template_path="ressource/test_dsl.
        yaml")
39      assert "Cycle d tecte" in str(excinfo.value)
40
41  def test_dsl_expression_invalide():
42      with pytest.raises(Exception) as excinfo:
43          solve("Methode avec formule invalide", {"x": 2}, template_path="ressource/
        test_dsl.yaml")
44      assert "Erreur lors de l'analyse" in str(excinfo.value)
45
46  def test_dsl_valide():
47      result = solve("Methode de test", {"a": 2, "b": 3}, template_path="ressource/
        test_dsl.yaml")
48      assert result["c"] == 5
49
50  def test_piecewise_n():
51      result = solve("Methode conditionnelle", {"x": -3}, template_path="ressource/
        test_dsl.yaml")
52      assert abs(result["f"] - 9) < 0.001
53
54  def test_piecewise_p():
55      result = solve("Methode conditionnelle", {"x": 4}, template_path="ressource/
        test_dsl.yaml")
56      assert abs(result["f"] - 4) < 0.001
57
58  def test_filtrage_basique():
59      values = {
60          "mf": 100,    # Moment flechissant relativement bas
61          "vf": 50,     # Effort tranchant relativement bas
62          "i": 200      # Inertie grande
63
64      result = filter_sections(values)
65      # Verifie qu'au moins un profile est retourner
66      assert not result.empty, "Aucun profile retourner"
67      # Verifie les contraintes respectees
68      assert all(result["Mr"] > 100), "Certains Mr sont inferieurs a mf"
69      assert all(result["Vr"] > 50), "Certains Vr sont inferieurs a vf"
70      assert all(result["Ix"] < 200), "Certains Ix sont superieurs a i"
71
72  def test_filtrage_aucun_resultat():
73      values = {"mf": 999999, "vf": 999999, "i": 1}
74      result = filter_sections(values)
75      assert result.empty, "Le filtrage devrait retourner aucun profile"
76
77  def test_solver_filtrage():
78      inputs = {"x": 1, "l": 10, "lt": 5, "cv": 1.5, "cm": 1.2}
79      result = solve("Poutre simple - Charge Uniformement Distribuee", inputs)
80      df = filter_sections(result)
81      assert not df.empty
82
83  def test_csv_contraintes_ok():
84      values = {"mf": 200, "vf": 300, "i": 100000}
```

```
    result = filter_sections(values, dataframe_path="ressource/test_sst12.csv")
    assert not result.empty, "Aucun profile retourne"
    assert all(result["Mr"] > values["mf"]), "Certains profiles ont un Mr trop faible"
    assert all(result["Vr"] > values["vf"]), "Certains profiles ont un Vr trop faible"
    assert all(result["Ix"] < values["i"]), "Certains profiles ont un Ix trop eleve"

def test_hauteur_et_poids_extraits():
    values = {"mf": 100, "vf": 50, "i": 200}
    result = filter_sections(values, dataframe_path="ressource/test_sst12.csv")
    assert "Hauteur_nominale" in result.columns
    assert "Poid_lineique" in result.columns
    assert result["Hauteur_nominale"].notna().all(), "Certaines hauteurs nominales sont manquantes"
    assert result["Poid_lineique"].notna().all(), "Certains poids lineiques sont manquants"

def test_tri_par_hauteur_et_poids():
    values = {"mf": 100, "vf": 50, "i": 200}
    result = filter_sections(values, dataframe_path="ressource/test_sst12.csv")
    # Tri croissant attendu
    sorted_result = result.sort_values(by=["Hauteur_nominale", "Poid_lineique"])
    assert result["Ds_m"].tolist() == sorted_result["Ds_m"].tolist(), "Le tri par hauteur/poids  n est  pas respecte"

def test_aucun_profil_si_contraintes_trop_strictes():
    values = {"mf": 99999, "vf": 99999, "i": 99999}
    result = filter_sections(values, dataframe_path="ressource/test_sst12.csv")
    assert result.empty, "Des profiles ont ete retournes alors  q u aucun  ne devrait satisfaire les contraintes"
```

Listing 5 – Test solver, DSL et filtrage

A.3.4 DSL de test

```
calculationOptions:
  Methode de test:
    - name: a
      label: a (unit)
    - name: b
      label: b (unit)

  Methode avec formule invalide:
    - name: x
      label: x (unit)

  Methode avec cycle:
    - name: x
      label: x (unit)

  Methode conditionnelle:
    - name: x
      label: x (m)

resultsTemplate:
  Methode de test:
```

```
    - name: c
      label: c (unit)

  Methode avec formule invalide:
    - name: y
      label: y (unit)

  Methode avec cycle:
    - name: z
      label: z (unit)

  Methode conditionnelle:
    - name: f
      label: f(x) (unit)

formules:
  Methode de test:
    c: "a + b"

  Methode avec formule invalide:
    y: "x **"  # Formule incorrecte

  Methode avec cycle:
    z: "w + 1"
    w: "z + 1"

  Methode conditionnelle:
    f: "Piecewise((x**2, x < 0), (x, x >= 0))"
```

Listing 6 – DSL de test

A.4. Documentation technique, schémas supplémentaires

A.4.1 Documentation technique du DSL

Le *Domain-Specific Language* (DSL) implémenté dans ce projet permet de décrire des méthodes de calcul structurales dans un format déclaratif, via des fichiers `.yaml`. L'objectif est de séparer la logique métier (équations, paramètres) du code source Python, afin de faciliter l'évolution et la maintenance du système.

Ce DSL est destiné à être utilisé par :

— des ingénieurs structuraux souhaitant définir ou adapter leurs méthodes de calcul ;

— des développeurs intégrant de nouvelles méthodes au système ;

Structure d'un fichier yaml Chaque fichier YAML décrivant une méthode de calcul comporte trois sections principales :

1. **`calculationOptions`** : paramètres d'entrée saisis par l'utilisateur.
2. **`resultsTemplate`** : variables de sortie affichées à l'utilisateur.
3. **`formules`** : équations calculées par le moteur.

Exemple minimal :

```
calculationOptions:
  Poutre simple - Charge Uniforme:
    - name: l
      label: Portée (m)
    - name: q
      label: Charge uniformément répartie (kN/m)

resultsTemplate:
  Poutre simple - Charge Uniforme:
    - name: mmax
      label: Moment max (kN·m)

formules:
  Poutre simple - Charge Uniforme:
    mmax: "q * l**2 / 8"
```

Règles de nommage et syntaxe

— `name` : identifiant interne (snake_case), sans espace.

— `label` : texte affiché à l'utilisateur (unités, accents et symboles permis).

— Les noms de variables dans les formules doivent correspondre aux identifiants `name` définis en entrée ou en sortie.

— Les formules sont évaluées via `SymPy` :

  — opérateurs standard : `+`, `-`, `*`, `/`, `**`

  — fonctions disponibles : `Max`, `Min`, `Piecewise`

  — conditions avec `&` (ET), `|` (OU), comparateurs `>`, `<`, `>=`, `<=`, `==`

Constantes globales : Certaines constantes (module d'élasticité E, gravité g, facteurs de charge, etc.) peuvent être définies dans fichier de `config.yaml` et réutilisées dans les formules.
Fonctions disponibles :

- `Max(a, b, ...)` : retourne le maximum
- `Min(a, b, ...)` : retourne le minimum
- `Piecewise((expr, cond), (expr2, cond2), ...)` : retourne la valeur selon condition

Exemple de `Piecewise` :

```
vd: "Piecewise((vf1 - wl * (x - a), (x > a) & (x < (a + b))),
               (0, True))"
```

Gestion des dépendances Le moteur (`solver.py`) applique un *tri topologique* pour évaluer les formules dans l'ordre correct.

- Si une formule dépend d'une variable non définie, une erreur est levée.
- Si un cycle de dépendances existe, la méthode est rejetée et une erreur est levée.

Bonne pratique : Même si l'ordre d'écriture n'est pas obligatoire, il est recommandé de placer les formules de manière logique pour faciliter la lecture.

Validation et tests Avant l'intégration d'une nouvelle méthode :

1. Vérifier que tous les paramètres sont utilisés.
2. S'assurer que toutes les variables sont définies.
3. S'assurer que les unités sont cohérentes.
4. Tester la méthode avec les jeux de données de `tests/test_dsl.py`.

Commande de test :

```
pytest test_dsl.py -k "ma_methode"
```

Exemples avancés

Poutre simple – Charge uniformément répartie :

```
formules:
  Poutre simple - Charge Uniformement Distribuée:
    wd_l: "(cm + cv) * lt"
    mf: "wl * (l * l) / 8"
    mx: "wl * x * (l - x) / 2"
    vf: "(wl * l) / 2"
    wl: "((cv * 1.5) + (cm * 1.25)) * lt"
```

Poutre simple – Charge concentrée au centre :

```
formules:
  Poutre simple - Charge Uniforme au Centre:
    wl: "((cv * 1.5) + (cm * 1.25)) * lt"
    vf1: "wl*b / (2*l) * (2*c + b)"
    mmax: "vf1 * (a + (vf1 / (2*wl)))"
```

Lexique des abréviations utilisées :

Nom	Description
`l`	Portée de la poutre (m)
`lt`	Largeur tributaire (m)
`cv`	Charge vive (kPa)
`cm`	Charge morte (kPa)
`mf`	Moment fléchissant (kN $\cdot$ m)
`mx`	Moment en x (kN $\cdot$ m)
`vf`	Effort tranchant (kN)
`wl`	Charge répartie pondérée (kN/m)

A.4.2 Lexique SST12

Types de sections

SHAPE_ID	Désignation	Description
1	W	Wide-flange shapes
2	HP	HP-shapes
3	M	M-shapes
4	S	S-shapes
5	C	Standard channels
6	MC	Miscellaneous channels
7	L	Single angles
8	WT	T-shapes cut from W-shapes
9	HS	Hollow structural sections – Square – CSA G40.20
10	HS	Hollow structural sections – Rectangular – CSA G40.20
11	HS	Hollow structural sections – Round – CSA G40.20
12	HA	Hollow structural sections – Square – ASTM A500
13	HA	Hollow structural sections – Rectangular – ASTM A500
14	HA	Hollow structural sections – Round – ASTM A500
15	2LE	Double angles – Equal legs back-to-back
16	2LL	Double angles – Long legs back-to-back
17	2LS	Double angles – Short legs back-to-back

(7)

Table 4 – Types de sections dans la base SST12

Définitions des abréviations

Abréviation	Définition
A_A6	Cross-sectional area, ASTM A6/A6M.
A_Th	Cross-sectional area, theoretical.
Avl	Availability : 'a' indicates a W-shape that is commonly used and readily available, although not produced in Canada. '*' indicates a section not produced in Canada. '+' indicates availability should be checked with supplier.
B	Width of flange or horizontal leg.
BetW	Symmetry parameter for unequal-leg angles in flexure about the major principal axis. See S16 :19 Clause 13.6.1(g).
BetX	Monosymmetry constant for WT-shapes ; positive when the flange is in flexural compression, and negative when the stem is in flexural compression. S16 :19 Clause 13.6.1(e)(ii).
BT	Width-to-thickness ratio of flange, wall or horizontal leg. S16 :19 Clause 11.3.
C	Torsion modulus, ratio of applied torque to maximum shear stress at outer face.
Crt	Shear constant, ratio of vertical shear force to maximum shear stress for the strong bending axis.
Cw	Warping constant.
D	Depth of section or height of vertical leg.
Dn_i	Nominal depth of section (Imperial units).
Dn_m	Nominal depth of section (Metric units).

Ds_i	Section designation (Imperial units).
Ds_m	Section designation (Metric units).
DT	Depth-to-thickness ratio of wall or vertical leg. See S16 :19 Clause 11.3.
DW	Depth-to-thickness ratio of stem of WT shapes. See S16 :19 Clause 11.3.
Grp	G40.20-13 shape size grouping for tensile properties. See G40.20-13 Standard.
HW	Clear depth-to-thickness ratio of web. See S16 :19 Clause 11.3.
Ix	Moment of inertia about axis XX.
Ixy	Product of inertia.
Iy	Moment of inertia about axis YY.
J	St-Venant torsional constant.
K	Distance between outer face of flange and web-toe of fillet.
K1	Distance between middle line of web and flange-toe of fillet.
Mass	Mass per length (Metric units).
Omeg	Buckling constant of asymmetric sections. See S16 :19 Clause 13.3.1.2.
Prp	Property group, '*' indicates the first section in a group with similar properties.
Qf	Statical moment at point in flange in line with web face.
Qw	Statical moment at mid-depth in web.
RI	Inner corner radius.
RO	Outer corner radius.
Rop	Polar radius of gyration about shear centre.
Rx	Radius of gyration about axis XX.
Rxp	Maximum radius of gyration about major principal axis (single angle).
Ry	Radius of gyration about axis YY.
Ryp	Minimum radius of gyration about minor principal axis (single angle).
Ry0	Radius of gyration about axis YY (0 mm back-to-back).
Ry8	Radius of gyration about axis YY (8 mm back-to-back).
Ry10	Radius of gyration about axis YY (10 mm back-to-back).
Ry12	Radius of gyration about axis YY (12 mm back-to-back).
Ry16	Radius of gyration about axis YY (16 mm back-to-back).
Ry20	Radius of gyration about axis YY (20 mm back-to-back).
R0	Radius of gyration about axis YY (0 in. back-to-back).
R1/4	Radius of gyration about axis YY (1/4 in. back-to-back).
R3/8	Radius of gyration about axis YY (3/8 in. back-to-back).
R1/2	Radius of gyration about axis YY (1/2 in. back-to-back).
R5/8	Radius of gyration about axis YY (5/8 in. back-to-back).
R3/4	Radius of gyration about axis YY (3/4 in. back-to-back).
SA	Surface area per length (m^2/m or ft^2/ft).
Shp	Shape identification code. See 'Contents' above.
Slp	Slope of tapered flange (tangent of angle).
Sw	Warping statical moment.
Sx	Elastic section modulus about axis XX.
Sy	Elastic section modulus about axis YY.
T	Nominal thickness of flange, leg or wall.
TanA	Tangent of rotation angle between geometric and principal member axes (single angle).

Tdes	Design thickness of wall for hollow structural sections. For HSS produced to ASTM A500, Tdes = 0.90 T.
T1	Thickness of tapered flange at tip.
T2	Thickness of tapered flange at flange-web junction.
Use	Normal use, 1-column, 2-column or Beam, 3-beam, 0-other.
W	Thickness of web.
Wn	Normalized unit warping function.
Wt_i	Weight per length (Imperial units).
X	Horizontal distance between centroid and outside face of web or vertical leg.
Xo	Distance between centroid and shear centre along geometric axis XX.
Xop	Distance between centroid and shear centre along major principal axis.
Y	Vertical distance between centroid and outside face of flange or horizontal leg.
Yo	Distance between centroid and shear centre along geometric axis YY.
Yop	Distance between centroid and shear centre along minor principal axis.
Zx	Plastic modulus about axis XX.
Zy	Plastic modulus about axis YY.

(7)

Table 5 – Définitions des abréviations SST12

A.4.3 Licence SST12 (CISC)

```
STRUCTURAL SECTION TABLES (SST12) LICENSE

This license agreement ("License") is for users who have obtained the Structural
Section Tables database spreadsheet ("SST12") from the Canadian Institute of
Steel Construction ("CISC").

YOUR AGREEMENT TO THIS LICENSE

This License only applies to the data on the dimensions and properties of
structural steel shapes contained in SST12. You should carefully read the
following terms and conditions before using SST12. Unless otherwise agreed in a
writing signed by an authorized representative of CISC, your use of SST12
indicates your acceptance of the License. You understand and agree that the
terms and conditions of this License are the only terms and conditions
applicable to your use of SST12, and any additional or different terms
specified, referenced, and/or incorporated by any document provided by you or on
your behalf are hereby rejected by CISC.

Please read the license terms below. If you do not agree to all of the terms and
conditions of this License, then do not use SST12.

The terms and conditions of this License describe the permitted use of each
Licensed Copy of SST12.

SCOPE OF LICENSE

Subject to the terms of this License, CISC grants to you, for each Licensed
Copy, a limited, non-exclusive, non-transferable, royalty-free license to use
SST12, solely for your personal or internal business purposes.

The SST12 spreadsheet is for your use alone. You shall not provide to any
person, and not permit any person to use or access SST12 or a copy of SST12. You
shall also not: (i) modify, translate, adapt or create derivative works based
upon SST12; (ii) develop, market, sell or distribute any product or service that
```

```
competes with CISC or includes features substantially similar to what is in
SST12; or (iii) use SST12 in any way that does not comply with all applicable
laws and regulations.

You may not give, transfer or sell copies of SST12.

All rights of any kind in SST12, which are not expressly granted in this
License, are entirely and exclusively reserved to and by CISC. SST12 is
protected by the Canadian Copyright Act and other applicable national and
international laws and treaties.

You may use SST12 solely as expressly provided in this License. You may not
rent, lease, loan, sublicense or modify SST12, in whole or in part, nor permit
anyone else to do so. You may not make access to SST12 available to others in
connection with a service bureau, application service provider, or similar
business, nor permit anyone else to do so.

WARRANTY DISCLAIMERS AND LIABILITY LIMITATIONS

Although no effort has been spared in an attempt to ensure that all the
dimensions and properties of structural steel shapes in this database are
factual, and that the numerical values are accurate to a degree consistent with
current structural design practice, SST12 is distributed and provided "as is"
and with no warranties of any kind, whether express or implied, including,
without limitation, any warranty of merchantability or fitness for a particular
purpose.

Neither CISC nor any of its affiliates warrant, guarantee, or make any
representations regarding the use of, or the results of the use of, SST12.
Neither CISC nor any of its affiliates warrants that the information contained
in SST12 will be error-free or complete. You acknowledge that good engineering
practice dictates that any information source, including SST12, must be
thoroughly tested with non-critical data before there is any reliance on it, and
```

```
you hereby assume the entire risk of all use of SST12 covered by this License.
This disclaimer of warranty constitutes an essential part of this License.

CISC and its affiliates shall not be liable for any damage or financial loss
arising out of the use of, or inability to use, SST12. In no event shall CISC,
or its principals, officers, employees, members, affiliates or contractors be
liable for any direct, indirect, incidental, consequential, special, or punitive
damages whatsoever relating to the use of SST12, or to your relationship with,
CISC and its affiliates (including, without limitation, loss of data or
information, loss of profit, revenue, business opportunity or business
advantage, or business interruption), whether based upon a claim or action of
contract, warranty, negligence, strict liability, contribution, indemnity, or
any other legal theory or cause of action, even if advised of the possibility of
such damages.

In addition, in no event does CISC authorize you or anyone else to use SST12 in
applications or systems where the failure of SST12 to perform can reasonably be
expected to result in a significant physical injury, or in loss of property, or
loss of life. Any such use is entirely at your own risk, and you agree to hold
CISC or its affiliates harmless from any and all claims or losses relating to
such unauthorized use.

GENERAL

This License is the complete statement of the agreement between the parties on
the subject matter and merges and supersedes all other or prior understandings,
purchase orders, agreements, and arrangements.

You may not assign, sublicense, subcontract, or otherwise transfer any of your
rights or obligations or delegate any of your duties under this License, in
whole or in part, without the prior written consent of CISC. Notwithstanding
```

```
anything to the contrary, any assignment of this License or any rights or
obligations hereunder is contingent upon the intended assignee agreeing to be
bound by the terms of this License and all responsibilities and obligations of
the assignor hereunder.

If any part of this License or the application thereof to any person or
circumstance is for any reason held invalid or unenforceable, it shall be deemed
severable, and the validity of the remainder of this License, or the
applications of such provision to other persons or circumstances, shall not be
affected thereby.
```